

GenAI & Retrieval-Augmented Generation

Practice Document for the AI Document Q&A; Project

Purpose: A deliberately substantial, text-based PDF for experimenting with PDF extraction, chunking, embeddings, vector search, and RAG. The content is fictional/educational and is designed to contain repeated concepts, headings, examples, and cross-references so retrieval experiments are meaningful.

Suggested first experiment: Run your `app/extract.py` script against this PDF and inspect the extracted text page by page. Do not build embeddings yet.

Learning sequence: PDF extraction → chunking → embeddings → similarity search → vector storage → retrieval → grounded generation → evaluation.

1. Generative AI Fundamentals

Generative artificial intelligence refers to systems that can produce new content from learned patterns. Depending on the model and task, generated content can include text, source code, images, audio, structured data, or combinations of these modalities. A language model generates text by repeatedly selecting tokens conditioned on the context supplied to it.

A useful distinction is between a model's learned knowledge and information supplied at inference time. A model may know broad facts from its training data, but an application may also need to answer questions about private documents, current internal policies, product manuals, or a recently uploaded report. Retrieval-Augmented Generation is one approach for supplying such external information at query time.

A basic generative AI application often has three conceptual layers. The first is the user interaction layer, where a person asks a question. The second is the orchestration layer, where the application prepares prompts, retrieves data, calls tools, and manages state. The third is the model layer, where an LLM generates a response. Production systems frequently add storage, authentication, monitoring, evaluation, and safety controls around these layers.

2. Large Language Models and Tokens

A large language model processes text as tokens rather than as complete words. A token may represent a whole short word, part of a word, punctuation, or another piece of text. The exact tokenization depends on the model. The model receives a sequence of tokens as context and predicts what token or tokens should come next.

The context window is the amount of tokenized information a model can consider for a request. If an application sends an entire long document for every question, it can consume substantial context even when most of the document is unrelated to the question. This creates practical tradeoffs involving latency, cost for hosted models, and the model's ability to focus on relevant evidence.

Conversation history is also context. If a program repeatedly appends previous user and assistant messages, every later request can contain more tokens. This is why the token count observed in a simple local LLM playground can increase as a conversation grows. RAG applies a related idea to documents: retrieve a small set of relevant text before generation rather than sending an entire corpus.

3. What Is RAG?

Retrieval-Augmented Generation, commonly abbreviated RAG, combines information retrieval with language-model generation. During ingestion, documents are processed, divided into chunks, embedded, and indexed. During a query, the user's question is embedded or otherwise searched against the index. The most relevant chunks are retrieved and placed into the model's context. The LLM then generates an answer using the retrieved evidence.

A basic RAG pipeline can be represented as: document → text extraction → cleaning → chunking → embedding → vector index. At query time the flow becomes: question → query embedding → similarity search → top-k chunks → prompt construction → LLM → answer. In a production system, additional steps such as metadata filtering, hybrid retrieval, reranking, query rewriting, citations, access control, and evaluation may be added.

RAG is not the same thing as fine-tuning. Fine-tuning changes model parameters for a training objective, while RAG keeps the model and supplies relevant external context at inference time. RAG is particularly useful when the knowledge source changes frequently or when the application needs to ground responses in a controlled document collection.

A good mental model is that the retriever acts like a librarian and the LLM acts like a writer. The librarian finds useful passages from the available collection. The writer uses those passages to compose a response. If the librarian retrieves poor evidence, even a capable writer may produce a poor grounded answer.

4. Document Ingestion

Document ingestion is the process of taking source material and preparing it for retrieval. Sources may include PDFs, text files, web pages, presentations, spreadsheets, databases, code repositories, or application records. Different sources require different extraction and normalization strategies.

For a text-based PDF, a library such as `pypdf` can expose pages and extract text. The extraction result should be inspected rather than blindly trusted. PDFs can contain multiple columns, tables, headers, footers, unusual reading order, images, or missing text layers. A scanned PDF may contain only page images and therefore require OCR before ordinary text-based chunking can work.

A useful ingestion pipeline separates extraction from later stages. Keeping the raw document, extracted text, normalized text, chunk metadata, and embedding records conceptually separate makes debugging easier. If retrieval quality is poor, an engineer can inspect each stage and determine whether the problem originated in extraction, chunking, embeddings, indexing, retrieval, or generation.

5. Text Cleaning and Normalization

After extraction, text may contain repeated headers, page numbers, excessive whitespace, broken lines, hyphenated words, or artifacts caused by PDF layout. Cleaning should be conservative. Removing too much information can damage meaning or remove useful headings and labels.

A simple first pass can normalize repeated spaces and line endings while preserving paragraph boundaries. More advanced pipelines may identify titles, headings, tables, lists, code blocks, and other structural elements. The goal is not to make text look perfect; the goal is to produce representations that support reliable retrieval.

Metadata is also important. A chunk can carry information such as document identifier, file name, page number, section title, creation date, access group, or source URL. Metadata can later be used to display citations, filter searches, or enforce access controls. For example, a retrieval system should not return a confidential chunk to a user who is not authorized to see it.

6. Chunking

Chunking divides a document into smaller retrieval units. A chunk should be large enough to preserve useful meaning but small enough that retrieval can return focused evidence. There is no universal chunk size that is optimal for every document type.

Fixed-size chunking splits text after a chosen number of characters or tokens. Fixed-size chunking is simple and useful as a baseline. Fixed-size chunking with overlap repeats a small amount of neighboring text between adjacent chunks so that information near a boundary is less likely to be separated completely.

Recursive or structure-aware chunking attempts to split at natural boundaries such as headings, paragraphs, sentences, or other separators before falling back to smaller units. Technical documents may benefit from preserving code blocks, tables, section titles, and list structures. Semantic chunking uses meaning or topic changes to decide where boundaries should occur.

Chunk size and overlap are retrieval parameters, not merely formatting choices. Very small chunks may lose context. Very large chunks may contain unrelated information and make retrieval less precise. Excessive overlap can increase storage and duplicate similar evidence. The right values should be evaluated against representative questions rather than chosen solely by convention.

Example: imagine a document section describing PostgreSQL connection pooling followed immediately by a section describing authentication. If the boundary splits a sentence or explanation, a small overlap can preserve enough neighboring context for either chunk to remain understandable. However, overlap should not become so large that every retrieved result contains mostly duplicated text.

7. Embeddings

An embedding is a numerical representation produced by an embedding model. Text with related meanings tends to occupy nearby regions of the model's vector space, although the exact behavior depends on the embedding model and the data. The resulting vector can have hundreds or thousands of dimensions.

The important point is that an embedding is not a human-readable definition of a sentence. It is a learned numerical representation used by downstream algorithms. A vector might look like [0.12, -0.43, 0.77, ...], but individual dimensions usually do not have simple human interpretations.

For RAG, document chunks are embedded during indexing. When a user asks a question, the query is embedded using the same embedding model. The system compares the query vector with stored chunk vectors. This allows retrieval to be based on semantic similarity rather than only exact keyword overlap.

The embedding model matters. Query and document vectors must be generated in a compatible vector space for meaningful similarity comparisons. If an indexed collection was created using one embedding model and the application later switches to an incompatible model, the old vectors generally need to be regenerated and reindexed.

8. Similarity Search

Similarity search ranks stored vectors according to how close they are to a query vector. Cosine similarity is a common measure. It compares the angle between vectors and is often convenient when vector magnitude is less important than direction.

Suppose a knowledge base contains five chunks. The query embedding is compared with each stored vector and produces scores such as 0.91, 0.23, 0.78, 0.41, and 0.86. The system can retrieve the top three chunks according to those scores. The exact score range and interpretation depend on the similarity metric and implementation.

Top-k retrieval means returning a fixed number of candidates, such as the top 3 or top 5 results. A production system may combine top-k with a minimum similarity threshold, metadata filters, keyword search, or a reranking stage. Retrieval quality should be measured using representative questions rather than assumed from scores alone.

9. Vector Databases and pgvector

A vector database or vector-capable database stores embeddings together with the original text and metadata and provides efficient similarity search. PostgreSQL can be extended with pgvector to store vector values and perform vector operations alongside ordinary relational data.

Using PostgreSQL for a RAG project is useful because an application can keep document metadata, users, permissions, chunk text, and vectors in a familiar database environment. A simplified record might contain `document_id`, `chunk_id`, `page_number`, `content`, `embedding`, and `created_at`.

A vector index improves the scalability of similarity search. For a learning project, however, it is valuable to first understand similarity calculations directly in Python before relying on a database abstraction. This makes it easier to understand what the vector database is actually doing.

10. Retrieval and Context Assembly

The retriever is responsible for turning a user question into useful evidence. A basic retriever embeds the query, searches the vector store, selects the top-k chunks, and returns them. The application then assembles a prompt containing instructions, retrieved context, and the original question.

A grounding prompt can tell the model to answer using the supplied context and to say that the information is unavailable when the context does not contain the answer. This does not guarantee perfect behavior, but it establishes an explicit generation objective.

Retrieved chunks should usually be accompanied by source metadata. If a response says that a fact came from page 7 of a document, the application should have a traceable relationship between the answer context and the original

source. This is useful for debugging and for user trust.

11. Generation with a Local LLM

A local LLM can generate the final response without sending the document contents to a hosted model. In this project, Ollama can act as the local model runtime and a Qwen-family model can be used for generation. The application supplies the retrieved context and question through the model's chat interface.

The model does not automatically know which parts of the context are trustworthy. The application should clearly delimit retrieved evidence and provide instructions about how to use it. A useful pattern is to separate system instructions, context, and question rather than mixing everything into one ambiguous paragraph.

Generation parameters also matter. A lower temperature may be appropriate when the goal is a repeatable factual answer, while a higher temperature can produce more variation. Temperature does not add knowledge to the model; it changes the sampling behavior used during generation.

12. Prompt Design for RAG

A RAG prompt can include an instruction such as: answer the question using only the provided context; if the context does not contain enough information, state that the document does not provide the answer. The prompt may also request concise answers, source references, or a structured response format.

Prompt templates are reusable because the question and retrieved context change for every request while the overall instruction remains stable. A template can contain placeholders such as {context}, {question}, and {source_information}. This is the same general idea as the prompt-template experiments in an LLM playground.

A robust prompt should avoid implying that retrieved text is automatically true in every situation. The application should distinguish between instructions and retrieved content, especially if documents can contain text that looks like system instructions. This helps reduce prompt injection risks from untrusted documents.

13. Citations and Traceability

A document Q&A; application becomes more useful when it can identify where an answer came from. A citation might include a document name, page number, section title, or chunk identifier. Citations should be derived from actual metadata stored with the retrieved chunks rather than guessed by the LLM.

Traceability also helps developers debug retrieval. If a user asks a question and the answer is wrong, the developer can inspect which chunks were retrieved. If the right passage was not retrieved, the problem is likely upstream in extraction, chunking, embeddings, filtering, or retrieval. If the right passage was retrieved but the answer ignored it, the problem may be in prompt design or generation.

14. Common RAG Failure Modes

RAG systems can fail in several different ways. Extraction failure occurs when important information is missing or corrupted before indexing. Chunking failure occurs when related information is separated or irrelevant content is grouped together. Embedding failure occurs when the representation does not capture the relationship needed for the query.

Retrieval failure occurs when the correct chunk is not among the retrieved candidates. Generation failure occurs when the LLM misinterprets, ignores, or adds unsupported information despite receiving relevant context. These failures can look similar to a user, so debugging requires examining intermediate stages.

Another common issue is stale data. If a source document changes but the vector index is not updated, retrieval may continue to return outdated content. A production ingestion process therefore needs a strategy for detecting changes and reindexing affected documents.

Access-control failures are particularly serious in enterprise systems. Retrieval must respect document and chunk permissions. It is not sufficient to hide a confidential source citation after retrieval if the confidential text has already been inserted into the model context.

15. Evaluation

A RAG application should be evaluated as a pipeline rather than judged only by whether the final answer sounds convincing. Retrieval evaluation asks whether the relevant evidence was retrieved. Generation evaluation asks whether the answer is supported by the retrieved evidence and whether it correctly answers the question.

A small evaluation dataset can contain questions, expected source passages, and expected answer characteristics. Metrics such as recall@k can help measure whether relevant chunks appear among the top k results. Human review can complement automated metrics, especially for nuanced questions.

Evaluation becomes particularly important when changing chunk size, overlap, embedding models, retrieval parameters, prompts, or reranking strategies. A change that sounds theoretically better may perform worse on the actual document collection. Reproducible test questions make these comparisons meaningful.

16. Advanced Retrieval Concepts

Basic vector similarity is only one retrieval strategy. Hybrid retrieval combines semantic vector search with lexical or keyword search. This can help when exact terms, identifiers, product names, error codes, or uncommon phrases matter. The two result sets can be combined and ranked.

Reranking is a second-stage process. Instead of asking an expensive reranker to inspect the entire corpus, the system first retrieves a manageable candidate set and then uses a stronger model or scoring method to reorder those candidates. This can improve precision while controlling latency.

Query rewriting transforms a user's original question into a search-oriented query. For example, a conversational follow-up such as 'What about its pricing?' may need the previous topic added before searching. Query rewriting can improve retrieval but adds another model call and therefore should be evaluated rather than assumed to help every task.

Metadata filtering can narrow retrieval before or alongside vector search. A question about a 2026 policy, for example, may benefit from a year filter. In enterprise systems, user permissions and document ownership can also become retrieval filters.

17. RAG Security

Documents can contain malicious or misleading instructions. A retrieved passage should normally be treated as data, not as a higher-priority instruction to the model. Prompt design should clearly distinguish application instructions from document content.

Sensitive documents require authorization checks before retrieval. The system should determine which sources a user is allowed to access and ensure that unauthorized chunks never enter the generation context. Logging should avoid exposing sensitive content unnecessarily.

Input and output validation can reduce operational risks. Applications can enforce maximum file sizes, allowed file types, query length limits, and reasonable retrieval counts. Generated answers can also be checked for required fields, citations, or policy constraints depending on the use case.

18. Observability and Debugging

A production RAG application should record useful operational information such as ingestion duration, document counts, chunk counts, embedding latency, retrieval latency, generation latency, and model identifiers. Logs should be

designed carefully so that private document contents are not exposed accidentally.

A trace for a single question can conceptually contain: request ID → query → retrieved chunk IDs → similarity scores → final prompt metadata → model response → latency. This makes it possible to investigate whether a problem is caused by retrieval or generation.

Monitoring can also reveal changes over time. If answer quality decreases after a document update, the index or extraction process may have changed. If latency increases, the number of retrieved chunks, embedding calls, database performance, or model generation speed may need investigation.

19. From Prototype to Production

A small learning project can run as a local Python script. A more complete application can expose document upload and question-answering endpoints through FastAPI. Authentication, database persistence, asynchronous processing, file validation, and background jobs can then be introduced as separate concerns.

A production architecture may contain an ingestion worker, object storage for original files, a relational database for metadata, a vector index for retrieval, an embedding service, an LLM service, and an API layer. The exact architecture depends on scale, privacy requirements, latency targets, and operational constraints.

Frameworks such as LangChain or LangGraph can later help organize complex workflows, but understanding the underlying pipeline first is valuable. A developer should be able to explain what extraction, chunking, embedding, retrieval, prompt construction, and generation are doing even when a framework hides implementation details.

20. Practical RAG Project Checklist

For the first implementation, keep the system deliberately simple. Start with a text-based PDF and verify extraction. Next implement a deterministic chunker and inspect the chunks. Then generate embeddings for the chunks and store them locally. Test similarity search with several questions before connecting an LLM.

After retrieval works, build the generation step. Pass only the top relevant chunks into the prompt and ask the model to answer from those chunks. Add source metadata so the application can show which pages were used. Then expose the pipeline through FastAPI and add PostgreSQL plus pgvector.

Finally, create a small evaluation set. Test questions that are directly answered by the document, questions requiring information across two sections, questions whose answer is absent, and questions using different wording from the source. This will teach you much more about RAG quality than testing only obvious keyword matches.

21. RAG Glossary

Term	Meaning
Chunk	A smaller piece of source content stored as a retrieval unit.
Embedding	A numerical vector representation generated from text by an embedding model.
Vector	An ordered collection of numerical values used to represent information mathematically.
Vector store	A storage and search system for embeddings and associated records.
Retriever	The component that searches indexed content and returns candidate evidence.
Top-k	A retrieval setting specifying how many highest-ranked candidates to return.
Similarity	A numerical measure indicating how closely two representations match under a chosen metric.
Cosine similarity	A vector similarity measure based on the angle between two vectors.
Reranking	A second-stage process that reorders an initial retrieval candidate set.
Grounding	Constraining or informing generation using external evidence supplied to the model.
Context	Information supplied to a model for a particular generation request.

Term	Meaning
Ingestion	The process of importing and preparing source data for indexing.
Metadata	Additional information associated with a chunk, such as page number or document ID.
OCR	Optical character recognition used to extract text from images or scanned documents.
Hybrid search	Combining different retrieval methods, commonly semantic and lexical search.
Query rewriting	Transforming a user's question into a more retrieval-friendly query.
Hallucination	Generated information that is unsupported, incorrect, or not grounded in available evidence.
Reindexing	Reprocessing source content and rebuilding its indexed representations.
pgvector	A PostgreSQL extension that adds vector data types and vector similarity operations.
RAG	Retrieval-Augmented Generation: retrieving external evidence and using it to augment LLM generation.

22. Retrieval Practice Questions

1. What is the main difference between RAG and fine-tuning?
2. Why can sending an entire 300-page document to an LLM be inefficient?
3. What happens during document ingestion?
4. Why is chunking performed before embedding?
5. What is an embedding?
6. Why should query and document embeddings normally come from the same compatible embedding model?
7. What does top-k mean in retrieval?
8. What is cosine similarity used for?
9. Why can chunk overlap help retrieval?
10. What is the tradeoff between very small and very large chunks?
11. What information could be stored as chunk metadata?
12. Why is page-number metadata useful?
13. What is the role of the retriever?
14. What is the role of the LLM after retrieval?
15. What is the difference between retrieval failure and generation failure?
16. Why does changing an embedding model generally require reindexing?
17. What is hybrid retrieval?
18. What is reranking?
19. Why can query rewriting help conversational questions?
20. Why must retrieval respect user permissions?
21. Why should retrieved document text be treated as data rather than instructions?
22. What is OCR and when is it needed?
23. Why is evaluation important when changing chunk size?
24. How can citations help debug a RAG system?
25. What operational metrics would you monitor in production?

23. Mini Case Studies

Employee Handbook

A company has a 250-page employee handbook. An employee asks about parental leave. A RAG system extracts and chunks the handbook, embeds the chunks, retrieves the passages about leave policies, and asks the LLM to answer using those passages. A production version should also attach policy year and access metadata.

Technical Documentation

A developer asks why an API request returns a particular error code. The knowledge base contains API documentation, troubleshooting guides, and release notes. Hybrid retrieval can be useful because exact error codes may be important while semantic retrieval can help when the developer paraphrases the problem.

Research Notes

A student uploads several papers and asks which sections discuss retrieval evaluation. A system can retrieve relevant chunks from multiple documents and cite the document title and page number. Evaluation questions can test whether the system retrieves evidence from the correct paper rather than merely returning passages containing the word 'retrieval'.

Confidential Policies

An enterprise assistant has public and confidential policy documents. The retrieval layer must apply authorization before confidential chunks are added to context. Hiding confidential citations after generation is not an adequate access-control mechanism because the model may already have received the confidential content.

24. End-to-End Mental Model

The simplest useful mental model for this project is: **Ingestion** prepares knowledge once; **retrieval** finds evidence for each question; **generation** turns the evidence into a useful response.

Ingestion: PDF → extraction → cleaning → chunks → embeddings → vector index.

Query: question → query embedding → similarity search → top-k chunks → context assembly → LLM → answer.

Production additions: metadata filters → permissions → hybrid retrieval → reranking → citations → evaluation → monitoring.

The key engineering lesson is that an RAG application is a pipeline. When something goes wrong, inspect the pipeline stage by stage instead of assuming the LLM is the only source of the problem.

25. Experiment Log Template

Experiment	Parameter / Change	Question	Observation	Result
1				
2				
3				
4				
5				
6				
7				
8				